

Enhancing xv6 with an MLFQ Scheduler

CSE323 Project Report - Group 8

Feature 1: Multi-Level Feedback Queue CPU Scheduling

1. Project Goals

This report describes the implementation and validation of Feature 1: a Multi-Level Feedback Queue (MLFQ) scheduler in xv6. The goal is to replace xv6's default round-robin scheduling behavior with an adaptive scheduler that reacts to process execution patterns.

The default xv6 scheduler scans the process table and runs RUNNABLE processes in a simple round-robin style. It does not distinguish between CPU-bound processes that consume long CPU bursts and I/O-bound or interactive processes that sleep frequently. The added MLFQ scheduler improves the kernel by giving short or interactive tasks higher priority while gradually moving CPU-heavy tasks to lower-priority queues.

Queue	Priority	Time Slice	Role
Queue 0	Highest	1 tick	New and interactive processes start here.
Queue 1	Medium	2 ticks	Processes demoted once run here.
Queue 2	Lowest	8 ticks	Long-running CPU-bound work runs here.

A starvation-prevention rule is also included: after every 100 global timer ticks, all processes are promoted back to Queue 0 and their per-queue tick counters are reset.

2. Modifications

The implementation required changes to process metadata, process initialization, scheduler selection, timer-interrupt handling, and user-level testing support. No new system call is required for Feature 1; existing xv6 mechanisms such as `fork()`, `sleep()`, `wait()`, timer interrupts, and `yield()` are sufficient to demonstrate the scheduler.

2.1 Kernel Data Structure Changes

The process structure was extended with two fields. The priority field records the current MLFQ queue, and ticks records how many CPU ticks the process has consumed in that queue.

```
// proc.h, inside struct proc
int priority; // current queue: 0, 1, or 2
int ticks; // ticks consumed at current priority
```

2.2 Process Initialization

Every newly allocated process begins in Queue 0 with zero consumed ticks. This ensures that new work initially receives the best response time.

```
// proc.c, inside allocproc(), after p->pid = nextpid++;
p->priority = 0;
p->ticks = 0;
```

2.3 Global Boost Counter

A global counter named `total` tracks elapsed scheduler ticks for priority boosting. It is defined in `proc.c` and referenced from `trap.c`.

```
// proc.c
int nextpid = 1;
int total = 0; // global counter for priority boosting

// trap.c
extern int total;
```

2.4 Priority-Based Scheduler

The scheduler was changed to scan Queue 0 first, then Queue 1, and finally Queue 2. A lower queue is considered only after the scheduler checks higher queues. Before each scan, the scheduler also checks whether the global boost threshold has been reached.

```

// proc.c, inside scheduler(), after acquire(&ptable.lock)
if(total >= 100){
    struct proc *pr;
    for(pr = ptable.proc; pr < &ptable.proc[NPROC]; pr++){
        pr->priority = 0;
        pr->ticks = 0;
    }
    total = 0;
    cprintf("MLFQ boost: all processes moved to queue 0\n");
}

for(q = 0; q < 3; q++){
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->state != RUNNABLE)
            continue;
        if(p->priority != q)
            continue;

        c->proc = p;
        switchvm(p);
        p->state = RUNNING;
        cprintf("pid %d running in queue %d\n", p->pid, p->priority);
        swtch(&(c->scheduler), p->context);
        switchkvm();
        c->proc = 0;
    }
}

```

2.5 Timer Interrupt Feedback and Demotion

Timer interrupts provide feedback about CPU usage. Each running process increments its own tick counter, and the global total counter is incremented for boosting. If a process uses its full queue time slice, it is demoted.

```

// trap.c, timer interrupt section
if(myproc() && myproc()->state == RUNNING &&
    tf->trapno == T_IRQ0+IRQ_TIMER){

    myproc()->ticks++;
    total++;

    if(myproc()->priority == 0 && myproc()->ticks >= 1){
        myproc()->priority = 1;
        myproc()->ticks = 0;
        yield();
    } else if(myproc()->priority == 1 && myproc()->ticks >= 2){
        myproc()->priority = 2;
        myproc()->ticks = 0;
        yield();
    } else if(myproc()->priority == 2 && myproc()->ticks >= 8){
        myproc()->ticks = 0;
        yield();
    }
}

```

3. Evaluation

The scheduler was evaluated with a user-level program named `mlfqtest.c`. The program uses `fork()` to create two different workloads. The parent behaves like an I/O-bound process by sleeping frequently, while the child behaves like a CPU-bound process by performing heavy computation.

```
// mlfqtest.c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    int pid = fork();

    if(pid < 0){
        printf(1, "fork failed\n");
        exit();
    }

    if(pid == 0){
        int i, j;
        printf(1, "CPU-bound child started\n");
        for(i = 0; i < 20; i++){
            for(j = 0; j < 100000000; j++){
            }
            printf(1, "CPU child loop %d\n", i);
        }
        exit();
    } else {
        int i;
        printf(1, "I/O-bound parent started\n");
        for(i = 0; i < 20; i++){
            printf(1, "Parent sleep loop %d\n", i);
            sleep(10);
        }
        wait();
        exit();
    }
}
```

Observed Output	Meaning
pid 4 running in queue 0 -> queue 1 -> queue 2	A CPU-bound process uses full slices and is demoted.
Parent sleep loop 0 ... 19	The parent repeatedly sleeps, modeling an I/O-bound process.
MLFQ boost: all processes moved to queue 0	The starvation-prevention boost fired after 100 global ticks.
pid 2 running in queue 0 after boost	A previously lower-priority process received high priority again.

These outcomes demonstrate the required behavior: CPU-bound work is demoted step by step, interactive or sleeping work remains responsive, and priority boosting prevents long-running low-priority processes from starving.

4. Conclusions

Feature 1 was completed by replacing the original round-robin style scheduling behavior with an MLFQ scheduler using three queues, time slices of 1, 2, and 8 ticks, CPU-usage based demotion, and 100-tick priority boosting. The implementation connects timer interrupts, process metadata, and scheduler selection into one observable scheduling policy.

A limitation of this implementation is that the debug printing is intentionally verbose for demonstration. A production-style scheduler would avoid printing during every scheduling decision and would expose statistics through a cleaner system call instead. Future improvements could include per-process wait-time accounting, a statistics syscall, and less intrusive evaluation output.

For final submission, this report should be submitted with diff_report.txt. The diff report must compare the original and modified xv6 folders and include only .c and .h file differences, as required by the project guideline.

Appendix: Diff Report Command

The required diff report can be generated from a temporary comparison repository after placing xv6-original and xv6-modified in the same parent directory.

```
mkdir compare_repo
cd compare_repo
git init

cp -r ../xv6-original/* .
git add .
git commit -m "original"

cp -r ../xv6-modified/* .
git add .
git commit -m "modified"

git diff HEAD~1 HEAD '*.c' '*.h' > ../diff_report.txt
```